

标准模板库

Standard Template Library, STL

教 师：冀荣华

单 位：人工智能系



主要内容

- 一、STL简介
- 二、容器
- 三、迭代器
- 四、常用算法



一、STL简介 标准模板库 Standard Template Library, STL

常用**数据结构**和**算法**的**模板**的集合。

包含三个基本组件：

- (1) **容器**：数据结构，如**vector**等，以模板类形式提供
- (2) **迭代器**：访问容器中对象的方法。相当于指针
访问容器中指定位置的元素
而不关心元素的具体类型
- (3) **算法**：操作容器中数据的**模板函数**



主要内容

一、STL简介

二、容器

三、迭代器

四、常用算法



二、容器

标准容器类

特点

顺序性容器

- vector
- deque
- list

从后面删除与插入，直接访问任何元素

从前面或后面删除与插入，直接访问任何元素

双链表，从任何地方删除与插入

元素在容器中的位置
同元素的值无关

关联容器

- set
- multiset
- map
- multimap

快速查找，不允许重复值

快速查找，允许重复值

关联容器内元素是排序的

一对多映射，基于关键字快速查找，不允许重复值

一对多映射，基于关键字快速查找，允许重复值

容器适配器

- stack
- queue
- priority_queue

后进先出

先进先出

最高优先级元素总是第一个出列



二、容器

//交换 int 变量的值

void Swap(int *a, int *b)

```
{  int temp = *a;  
    *a = *b;  
    *b = temp;}
```

//交换 float 变量的值

void Swap(float *a, float *b)

```
{  float temp = *a;  
    *a = *b;  
    *b = temp;}
```

//交换 char 变量的值

void Swap(char *a, char *b)

```
{  char temp = *a;  
    *a = *b;  
    *b = temp;}
```

template<typename T>

void Swap(T *a, T *b)

```
{  
    T temp = *a;  
    *a = *b;  
    *b = temp;  
}
```



二、容器

```
#include <iostream>
using namespace std;
template<typename T>
void Swap(T *a, T *b)
{   T temp = *a;
    *a = *b;
    *b = temp;}
void main()
{   //交换 int 变量的值
    int n1 = 100, n2 = 200;
    Swap(&n1, &n2);
    cout<<n1<<" "<<n2<<endl;
    //交换 char 变量的值
    char c1 = 'A', c2 = 'B';
    Swap(&c1, &c2);
    cout<<c1<<" "<<c2<<endl;
}
```



二、容器

template <typename 类型参数1 , typename 类型参数2 , ...>

返回值类型 函数名(形参列表)

```
{  
    //TODO:  
}
```

template<typename 类型参数1 , typename 类型参数2 , ...>

class 类名

```
{  
    //TODO:  
};
```

ClassName <类型> 对象名;

vector<int> a;



二、容器

```
#include "iostream.h"
template<typename T1, typename T2>
class Point
{public:
    Point(T1 x, T2 y): m_x(x), m_y(y)
        { }
    T1 getX() const;
    void setX(T1 x);
    T2 getY() const;
    void setY(T2 y);
private:
    T1 m_x; //x坐标
    T2 m_y; //y坐标
};
template<typename T1, typename T2> //模板头
T1 Point<T1, T2>::getX() const /*函数头*/
{    return m_x;}
```



二、容器

```
template<typename T1, typename T2>
```

```
void Point<T1, T2>::setX(T1 x)
```

```
{    m_x = x;}
```

```
template<typename T1, typename T2>
```

```
T2 Point<T1, T2>::getY() const
```

```
{    return m_y;}
```

```
template<typename T1, typename T2>
```

```
void Point<T1, T2>::setY(T2 y)
```

```
{    m_y = y;}
```

```
void main()
```

```
{    Point<int, int> p1(10, 20);
```

```
    cout<<"x="<<p1.getX()<<"", y="<<p1.getY()<<endl;
```

```
    Point<int, char*> p2(10, "东经180度");
```

```
    cout<<"x="<<p2.getX()<<"", y="<<p2.getY()<<endl;
```

```
    Point<char*, char*> *p3 = new Point<char*, char*>("东经180度", "北纬210度");
```

```
    cout<<"x="<<p3->getX()<<"", y="<<p3->getY()<<endl;
```

```
    delete p3;
```

```
}
```



二、容器

容器都是类模板

实例化后就成为容器类

用容器类定义的对象称为容器对象

vector<int>——容器类名字

vector<int> a; //定义容器对象 a，a 代表长度可变的数组，每个元素是 int 变量



二、容器

任何两个容器对象，只要类型相同，就可用 $<$ 、 $<=$ 、 $>$ 、 $>=$ 、 $==$ 、 $!=$

假设 a 、 b 是两个类型相同的容器对象，运算规则如下：

$a == b$: 若 a 和 b 中的元素个数相同，且对应元素均相等，则为 `true`，否则为 `false`

$a < b$: 从头到尾依次比较每个元素，如 a 中元素小于 b 中元素，则为 `true`;

$a != b$: 等价于 $!(a == b)$

$a > b$: 等价于 $b < a$

$a <= b$: 等价于 $!(b < a)$

$a >= b$: 等价于 $!(a < b)$

所有容器都有成员函数：

`int size()`：返回容器对象中元素的个数。

`bool empty()`：判断容器对象是否为空。

顺序容器和关联容器还有以下成员函数：

`begin()`：返回指向容器中第一个元素的迭代器

`end()`：返回指向容器中最后一个元素后面的位置的迭代器

`rbegin()`：返回指向容器中最后一个元素的反向迭代器

`rend()`：返回指向容器中第一个元素前面的位置的反向迭代器

`erase(...)`：从容器中删除一个或几个元素

`clear()`：从容器中删除所有元素

如果容器是空的，则 `begin()` 和 `end()` 的返回值相等，`rbegin()` 和 `rend()` 的返回值也相等。

二、容器

顺序容器还有以下常用成员函数：

`front()`：返回容器中第一个元素的引用

`back()`：返回容器中最后一个元素的引用

`push_back()`：在容器末尾增加新元素

`pop_back()`：删除容器末尾的元素

`insert(...)`：插入一个或多个元素



二、容器

向量 **vector**

相当于数组，其大小可不预先指定，且自动扩展——动态数组
能够存放任意类型的动态数组

特点：

- (1) 指定一块可动态扩展连续存储空间，
- (2) 随机访问方便，可以采用[] 操作符和vector.at()
- (4) vector只能在尾部追加和删除操作
- (5) 只能在vector 的尾部进行push 和pop
- (6) 当动态添加的数据超过vector 默认分配的大小时要进行内存的重新分配、拷贝与释放，这个操作非常消耗性能。所以要vector 达到最优的性能，最好在创建vector 时就指定其空间大小。



二、容器

vector的使用

1、使用vector，在代码中须包含下面的代码：

```
#include <vector>
```

```
using namespace std;
```

然后在程序中就可以使用语句

```
vector<int> v1;
```

或

```
#include <vector>
```

然后在程序中就可以使用语句

```
std::vector<int> v1;
```



vector的创建方法:

- 创建一个T类型的空~~空~~的vector对象:

```
vector<T> tv;
```

- 创建一个包含500个~~个~~T类型数据的tv:

```
vector<T> tv(500);
```

- 创建一个包含500个~~个~~T类型数据的vector, 并且都~~初始化~~为0:

```
vector<T> tv(500, T(0));
```

- 创建一个T的~~拷贝~~:

```
vector<T> tv2(tv);
```



二、容器

```
#include<iostream.h>
```

```
#include<vector>
```

```
using namespace std;
```

```
void main()
```

```
{    vector<int> c(5,0);
```

```
    int i;
```

```
    for(i=0;i<5;i++)
```

```
    {    c[i]=i;
```

```
        cout<<c[i]<<endl;
```

```
    }
```

```
    cout<<c.size()<<endl;//返回容器中实际数据的个数
```

```
}
```



二、容器

```
#include<iostream.h>
#include<vector>
using namespace std;
void main()
{
    vector<int> c;
    int i;
    c.assign(10,5); //10个5赋给c
    for(i=0;i<10;i++)
        cout<<c[i]<<endl;
    cout<<endl;
    cout<<c.size()<<endl;
}
```



二、容器

```
#include<iostream.h>
#include<vector>
using namespace std;
void main()
{
    vector<int> c(20);
    int i;
    c.assign(10,5);
    for(i=0;i<c.size();i++)
    {
        c[i]=5-i;
        cout<<c[i]<<endl;
    }
    c.push_back(100);//尾部添加一个数据100
    i=c.at(0);//将下标为0的数据赋给i
    //i=c.front();//返回第一个数据
    //i=*(c.begin());//返回第一个数据
    //      i=c.back(); //返回最后一个数据
    cout<<i<<endl;
}
```



二、容器

```
#include<vector>
#include<iostream>
using namespace std;
void main()
{
    vector<int> v(3);
    int i;
    v[0]=2;          v[1]=7;    v[2]=9;
    for(i=0;i<v.size();i++)
        cout<<v[i]<<" ";
    cout<<endl;
    v.insert(v.begin(),8);//在最前面插入新元素
    for(i=0;i<v.size();i++)
        cout<<v[i]<<" ";
    cout<<endl;
    v.insert(v.begin()+2,1);//在第二个元素前插入新元素
    for(i=0;i<v.size();i++)
        cout<<v[i]<<" ";
    cout<<endl;
    v.insert(v.end(),3);//在向量末尾追加新元素。
    for( i=0;i<v.size();i++)
        cout<<v[i]<<" ";
    cout<<endl;    }
```



```
#include "iostream.h"
#include "vector"
using namespace std;
void main()
{
    int i,b[7]={1,2,3,4,5,6,7};
    vector<int>c(&b[1],&b[7]);
    for(int i=0;i<c.size();i++)
        cout<<c[i]<<endl;
    c.push_back(12);
    c.insert(&c[4],13);
    for(i=0;i<8;i++)
        cout<<c[i]<<endl;
    cout<<*c.begin()<<'\t'<<*(c.end()-2)<<'\t'<<*c.rbegin()<<'\t'<<*(c.rend())<<endl;
}
```

二、容器

```
#include<iostream.h>
#include<vector>
using namespace std;
void main ()
{
    vector<int>c(5,123);
    int i;
    int temp;
    cin>>temp;
    c.push_back(temp);
    c.insert(c.begin()+2,12);
    for(i=0;i<7;i++)
    {
        cout<<c[i]<<endl;    }
    cout<<c.empty ()<<endl;
    c.erase(c.begin(),c.end());
    cout<<c.empty ()<<endl;
}
```



二、容器

```
#include "iostream"
#include <vector>
using namespace std;
void main()
{
    vector<int>c(3,1);        int i;
    cout<<"the original vector:\n";
    for(i=0;i<5;i++)
        cout<<c[i]<<endl;
    cout<<"the size is : "<<c.size ()<<endl;
    cout<<"the end data is : "<<c.back ()<<endl;
    cout<<"the first data is : "<<c.front ()<<endl;
    int s[]={6,66,666};
    c.insert(c.end() ,s,s+3);
    cout<<"the new vector:\n";
    for(i=0;i<c.size ();i++)
        cout<<c[i]<<endl;
    c.clear ();
    cout<<"the vector is empty? ";
    cout<<c.empty ()<<endl;
    cout<<"the last new vector:\n";
    for(i=0;i<8;i++)
        cout<<c[i]<<endl; }
```



二、容器

```
#include<vector>
#include<iostream>
using namespace std;
void main()
{
    vector<int> v(3);
    int i;
    v[0]=2;
    v[1]=7;
    v[2]=9;
    for(i=0;i<v.size();i++)
        cout<<v[i]<<" ";
    cout<<endl;
    //v.erase(v.begin());
    //v.pop_back();
    v.erase(v.end()-1);
    for( i=0;i<v.size();i++)
        cout<<v[i]<<" ";
    cout<<endl;
}
```



主要内容

一、STL简介

二、容器

三、迭代器

四、常用算法



三、迭代器

迭代器： 相当于指针

```
int a = 10;
```

```
int *p;
```

```
p = &a;
```

```
*p = 11; // 操作后a 变为11
```



三、迭代器

指针===地址

用来存放指针的变量就是指针变量

```
int a;
```

```
int * p;
```

```
p=&a;
```

等价于

```
int a;
```

```
int * p=&a;
```

说明：p指向了a

以后程序中的*p读作：p所指向的变量

所以后面的程序中*p等价于a

在程序中使用*p之前，须明确p中存放的是谁的地址，即p指向了谁



三、迭代器

迭代器定义方式

1) 正向迭代器:

容器类名::**iterator** 迭代器名;

2) 常量正向迭代器:

容器类名::**const_iterator** 迭代器名;

3) 反向迭代器:

容器类名::**reverse_iterator** 迭代器名;

4) 常量反向迭代器:

容器类名::**const_reverse_iterator** 迭代器名;



三、迭代器

begin 成员函数返回指向容器中第一个元素的迭代器

`++ip` 使得 `ip` 指向容器中的下一个元素

end 成员函数返回的是指向最后一个元素后面的位置的迭代器

反向迭代器进行`++`操作后，会指向容器中的上一个元素。

rbegin 成员函数返回指向容器中最后一个元素的迭代器

rend 成员函数返回指向容器中第一个元素前面的位置的迭代器

容器适配器 `stack`、`queue` 和 `priority_queue` 没有迭代器



三、迭代器

```
#include<vector>
#include<iostream>
using namespace std;
void main()
{
    vector<int> c;
    for(int i =0;i<5;i++)
    {
        c.push_back(i);
        cout<<c[i]<<" ";}
    cout<<endl;
    // 定义一个迭代器
    vector<int>::iterator p;
    // 指向容器的首个元素
    p=c.begin();
    // 移动到下一个元素
    p++;
    // 修改该元素赋值
    *p = 20 ;//则c 容器中的第二个值被修改为了20
    for(i =0;i<5;i++)
    {
        cout<<c[i]<<" ";}
}
```



三、迭代器

```
#include<vector>
#include <iostream>
using namespace std;
void main()
{
    vector<int> arr;
    arr.push_back(6);
    arr.push_back(8);
    arr.push_back(3);
    arr.push_back(8);
    vector<int> ::iterator it;
    for(it=arr.begin(); it!=arr.end(); )
    {
        if(* it == 8)
            it = arr.erase(it);
        else
            ++it;
    }
    cout << "After remove 8:\n";
    for(it = arr.begin();it<arr.end(); ++it)
        cout << * it << " ";
    cout << endl;
}
```



三、迭代器

```
#include <iostream>
#include <vector>
using namespace std;
void main()
{ vector<int> v;
  for (int n = 0; n<5; ++n)
    v.push_back(n);
  vector<int>::iterator i;
  for (i = v.begin(); i != v.end(); ++i)
  {
    cout << *i << " "; // *i 就是迭代器i指向的元素
    *i *= 2; // 每个元素变为原来的2倍
  }
  cout << endl;
  for (vector<int>::reverse_iterator j = v.rbegin(); j != v.rend(); ++j)
    cout << *j << " ";
}
```



三、迭代器

不同容器的迭代器的功能

容器	迭代器功能
vector	随机访问
deque	随机访问
list	双向
set / multiset	双向
map / multimap	双向
stack	不支持迭代器
queue	不支持迭代器
priority_queue	不支持迭代器



三、迭代器

STL 中有用于操作迭代器的三个函数模板：

advance(p, n)：使迭代器 **p** 向前或向后移动 **n** 个元素。

distance(p, q)：计算两个迭代器之间的距离，即迭代器 **p** 经过多少次 ++ 操作后和迭代器 **q** 相等。如果调用时 **p** 已经指向 **q** 的后面，则会陷入死循环

iter_swap(p, q)：用于交换两个迭代器 **p**、**q** 指向的值。



三、迭代器

```
#include <list>
#include <iostream>
#include <algorithm>
using namespace std;
void main()
{ int a[5] = { 1, 2, 3, 4, 5 };
  list<int> lst(a, a+5);
  list<int>::iterator p = lst.begin();
  for(int i = 0; i < lst.size() ; ++i)
    cout << *(p++)<<"\t";
  advance(p, 2); //p向后移动两个元素, 指向3
  cout << "\n1)" << *p << endl; //输出 1)3
  advance(p, -1); //p向前移动一个元素, 指向2
  cout << "2)" << *p << endl; //输出 2)2
  list<int>::iterator q = lst.end();
  q--; //q 指向 5
  cout << "3)" << distance(p, q) << endl; //输出 3)3
  iter_swap(p, q); //交换 2 和 5
  cout << "4)";
  for (p = lst.begin(); p != lst.end(); ++p)
    cout << *p << " ";
}
```



```
#include<stdio.h>
#include<algorithm>
#include<vector>
#include<iostream>
using namespace std;
class Rect
{public:
    int id;
    int length;
    int width;

}
};
```

```
void main()
{ vector<Rect> vec;
  Rect rect;
  rect.id=1;    rect.length=2;    rect.width=3;
  vec.push_back(rect);
  vector<Rect>::iterator it=vec.begin();
  cout<<(*it).id<<' '<<(*it).length<<' '<<(*it).width<<endl;
}
```

https://blog.csdn.net/u010183728/article/details/81913729?utm_medium=distribute.pc_relevant_download.none-task-blog-2~default~BlogCommendFromBaidu~default-1.nonecase&depth_1-utm_source=distribute.pc_relevant_download.none-task-blog-2~default~BlogCommendFromBaidu~default-1.nonecas#t0

主要内容

- 一、STL简介
- 二、容器
- 三、迭代器
- 四、常用算法



四、常用算法

通用算法（约有70种），如插入、删除、查找、排序等。

算法就是函数模板。

算法通过迭代器来操纵容器中的元素。

算法涉及头文件 **algorithm**、**numeric**、**functional**

有的算法会改变其所作用的容器。例如：

- copy：将一个容器的内容复制到另一个容器
- remove：在容器中删除一个元素
- random_shuffle：随机打乱容器中的元素
- fill：用某个值填充容器

有的算法不会改变其所作用的容器。例如：

- find：在容器中查找元素
- count_if：统计容器中符合某种条件的元素的个数

<https://www.cnblogs.com/linuxAndMcu/p/10264339.html>



四、常用算法

```
#include <vector>
#include <algorithm>
#include <iostream>
using namespace std;
void main()
{   int a[10] = {10,20,30,40};
    vector<int> v;
    v.push_back(1);   v.push_back(2);
    v.push_back(3);   v.push_back(4); //此后v里放着4个元素: 1,2,3,4
    vector<int>::iterator p;
    p = find(v.begin(),v.end(),3); //在v中查找3
    if(p != v.end()) //若找不到,find返回 v.end()
        cout << "1) " << * p << endl; //找到了
    int * pp = find(a,a+4,20);
    if(pp == a + 4)
        cout << "not found" << endl;
    else
        cout << "2) " << * pp << endl;   }
```

应用实例

5个选手，10个评委

10个评委给每个选手打分，去掉最高分和最低分，再取平均值为最终得分

最后再给每个选手按分数排序



应用实例

```
#include "stdio.h"
#include <string>
#include <iostream>
#include <vector>
#include <deque>
#include <algorithm>
using namespace std;
class Player
{public:
    Player(string name,int score):m_score(score),m_name(name)
    {
    }
    int m_score;
    string m_name;
};
```



应用实例

//构造选手

```
void SetPlayer(vector<Player> & player)
{
    string str1 = "ABCDE";
    for (int i = 0; i < 5; i++)
    {
        string str2 = "Player";
        str2 +=str1[i];
        Player p(str2, 0); //PlayerA PlayerB.....
        player.push_back(p);
    }
}
```

```
bool mycompar(Player& p1, Player& p2) //比较
{
    return p1.m_score < p2.m_score;}
```

//排序

```
void SortPlayer(vector<Player>& player)
{
    sort(player.begin(), player.end(), mycompar);}
void print(vector<Player>& player)
{
    for (vector<Player>::iterator it = player.begin(); it != player.end(); it++)
    { cout << "Name:" << (*it).m_name << "    " << "Scores:" << (*it).m_score << endl;}
}
```



应用实例

// 打分

```
void SetScore(vector<Player> & player)
{
    for (vector<Player>::iterator it = player.begin(); it != player.end(); it++)
    {
        deque<int> deq_s;
        for (int j = 0; j < 10; j++) //10个评委打分
        {
            int scorexx = rand() % 41 + 60; //60-100
            deq_s.push_back(scorexx);
        }
        sort(deq_s.begin(), deq_s.end()); //排序
        deq_s.pop_back(); //去掉最高分和最低分
        deq_s.pop_front();
        int sum=0;
        for (deque<int>::iterator itd = deq_s.begin(); itd != deq_s.end(); itd++)
        {
            sum += (*itd);
        }
        int average=sum / deq_s.size(); //求平均值
        (*it).m_score = average; }

void main()
{
    vector<Player> player;
    SetPlayer(player);           SetScore(player);
    SortPlayer(player);          print(player);}
```



方法二:

```
#include <iostream>
#include <string>
#include <vector>
#include <deque>
#include <stdlib.h> //srand rand
#include <time.h> //time
#include <algorithm> //sort
                                #include <numeric>

using namespace std;
class Person
{
    string m_name;    int m_score;
public:
    Person(){}
    Person(string name, int score)
    { m_name = name;    m_score = score; }
    void showPerson()
    { cout<<"姓名:"<<m_name<<", 分数:"<<m_score<<endl; }
    void setScore(int score)
    { m_score = score; }
};
```



```
void ceatePlayer(vector<Person> &v)
```

```
{ int i=0;
```

```
    string tmp="ABCDE";
```

```
    for(i=0;i<5;i++)
```

```
    { string name="选手";
```

```
        name += tmp[i];
```

```
        v.push_back(Person(name, 0));    }
```

```
}
```

```
void printVectorPerson(vector<Person> &v)
```

```
{ vector<Person>::iterator it=v.begin();
```

```
    for(;it!=v.end();it++)
```

```
    { (*it).showPerson(); }
```

```
}
```



```
void playGame(vector<Person> &v)
{ srand(time(NULL));
  vector<Person>::iterator it=v.begin();
  for(; it!=v.end();it++)
  { /*it代表每位选手,deque存放评委的分数
    deque<int> d;
    int i=0;
    for(i=0;i<10;i++)
    { int score = 0;
      score = rand()%41+60;//60~100
      d.push_back(score);
    }
    sort(d.begin(), d.end());
    d.pop_back();
    d.pop_front();
    int sum = accumulate(d.begin(), d.end(), 0);
    int avg = sum/d.size();
    (*it).setScore(avg);
  }
}
```




```
void main()
{
    vector<Person> v;
    //创建5名选手
    ceatePlayer(v);
    //遍历容器
    printVectorPerson(v);
    //参加比赛
    playGame(v);
    cout<<"-----"<<endl;
    //遍历容器
    printVectorPerson(v);
}
```



请将电院学生成绩管理**C版本**改为**C++版本**

- 1) 将与成绩相关数据、函数，放到一起生成成绩类
- 2) 实现与C版本一致的功能
- 3) 需要将STL应用在本程序中
- 4) 进一步的实现多门成绩管理

<https://www.cnblogs.com/linuxAndMcu/p/10264339.html>

<https://www.apiref.com/cpp-zh/cpp/header/numeric.html>